National and Kapodistrian University of Athens
Department of Informatics and Telecommunications

Deep Learning for NLP

Student name: *Margarita Orfanidi*
ID: 7115152400023

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

Contents

1. Abstract

The identification of ambiguity in the responses politicians give during interviews is a highly challenging task in the field of NLP. The goal is to classify political responses into predefined categories that reflect the level of clarity or evasiveness present in an answer. Based on the dataset released for SemEval 2026 Task 6 (CLARITY), this study explores this NLP problem and develops two classification models. Both models employ Logistic Regression as the underlying classification algorithm, but they differ in their text representation methods. The first approach represents text using TF-IDF features, while the second utilizes Word2Vec embeddings. The experiments focus on evaluating the performance of each method individually, as well as comparing the two feature extraction techniques within the same classification framework.

2. Analysis

2.1. Dataset

The dataset consists of a collection of questions and answers from Presidents of the United States during interviews. The dataset is divided into two parts: a training set and a test set. The training set contains 3,448 entries with 20 attributes, while the test set contains 308 entries with the same attributes. The columns are described in the following table.

Column	**Description**
title	Interview title
date	Date of the interview
president	President being interviewed
url	Source URL of the interview
question_order	Order of the question in the interview
interview_question	Original interview question
interview_answer	Answer given by the president
gpt3.5_summary	Summary generated by GPT-3.5
gpt3.5_prediction	Prediction generated by GPT-3.5
question	Sub question
annotator_id	Identifier of the annotator
annotator1	Label from annotator 1
annotator2	Label from annotator 2
annotator3	Label from annotator 3
inaudible	Indicates if the segment contains inaudible parts
multiple_questions	Indicates if the question contains multiple parts
affirmative_questions	Indicates if the question is affirmative
index	Entry index
clarity_label	Label indicating response clarity
evasion_label	Label indicating response evasion

Table 1: Columns of the SemEval 2026 Task 6 dataset.

The dataset includes textual components such as the original interview question,

the sub-questions, and the corresponding interview response. Moreover, it contains metadata, such as the interview title, date, and the name of the president. Finally, the dataset includes the proposed classification labels provided at the sub-question level, with each sub-question assigned classification labels (clarity_label and evasion_label) based on the proposed taxonomy.

It is important to note that the training set does not contain the individual labels assigned by each annotator, but only the final aggregated labels. Therefore, although the dataset generally contains non-null values, all values in these specific columns are null.

In contrast, in the test set, most metadata fields are null, while the columns necessary for evaluation are not null, such as the main text columns, the individual annotator labels (annotator1, annotator2, annotator3), and the corresponding labels.

2.2. Exploratory Data Analysis

Before building an accurate classifier, a study of the data and the nature of the problem must first be conducted. It is necessary to examine basic statistics, understand the distribution of the labels, and identify potential patterns and correlations in the data. Through this process, we can confirm the structure of the dataset, recognize the most important variables, and develop a clearer understanding of the problem we are trying to solve.

2.2.1. Samples. Random samples were displayed from the dataset in order to observe the structure and formatting of the textual data. This qualitative inspection helps identify potential patterns and understand how the text appears in practice, allowing us to draw conclusions about the preprocessing steps that may be required. For example, in the dataset, several spelling inconsistencies, missing spaces, and the presence of punctuation marks were observed, creating an initial intuition about the type of preprocessing that may be needed.

Sample 2900

	Value
title	The President's News Conference With President Lee Myung-bak of South Korea at Camp David, Maryland
date	April 19, 2008
president	George W. Bush
url	https://www.presidency.ucsb.edu/documents/the-presidents-news-conference-with-president-lee-myung-bak-south-korea-camp-david
question_order	2
interview_question	Q. I have a question for President Lee. Korea and the United States have made many achievements through the summit meeting, especially North Korean nuclear issue and the strengthening of the alliance. As for North Korean nuclear issue, Mr. President Lee suggested setting up a permanent liaison office in both Seoul and Pyongyang. What are some of the follow-up effects, if you do have any follow-up actions? And do you have any thoughts of proposing a meeting with Chairman Kim at an earlier date?
interview_answer	The process is not something that we discussed between ourselves during the summit meeting. In fact, when I was staying in Washington, DC, I had an interview with one of the newspapers there, and it came up. Of course, it was not a sudden suggestion. I did have a meeting among my staff and related ministries, and I talked about this in detail before I came to the United States.We have a new administration in Korea, and we haven't yet to begun dialogue with the ns. Inter-Korean dialogue, there is a need for us to have dialogue all the time. Up until now, we had dialogues whenever the need arose, and then it would stop. However, dialogue should be based on genuine cooperation and sincerity. And so with this in mind, I thought that it would be helpful to set up a permanent liaison office in both Seoul and Pyongyang.As for the summit meeting between myself and Chairman Kim, I will agree to it when the need is real. And I already said publicly that I am willing to meet with him—not just once, but many times—but if the meeting will yield substantial and real results. I believe only when that is possible, I am ready to meet with him and have sincere dialogue, because that will help to bring about peace and stability of the peninsula.So basically, I do hold that thought, but I'm not suggesting that—to have a meeting with Chairman Kim anytime soon. If the need arises, again, I'm ready to meet with him. Steven Lee [Steven Lee Myers, New York Times].
gpt3.5_summary	The question consists of 2 parts:\n\n1. Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.\n2. Thoughts on proposing a meeting with Chairman Kim at an earlier date.\n\n<--Summary Answer-->\n\nThe response provides the following information regarding these points:\n\n1. Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue:\n- President Lee suggests setting up a permanent liaison office in both Seoul and Pyongyang to facilitate dialogue and cooperation between North Korea and South Korea.\n- The suggestion was made during an interview with a newspaper in Washington, DC, and was not discussed during the summit meeting.\n- The intention behind the suggestion is to establish genuine cooperation and sincerity in inter-Korean dialogue.\n\n2. Thoughts on proposing a meeting with Chairman Kim at an earlier date:\n- President Lee is open to the idea of a meeting with Chairman Kim, but only when the need is real and there is a possibility of substantial and real results.\n- He believes that sincere dialogue with Chairman Kim can contribute to the peace and stability of the Korean peninsula.\n- While he holds the thought of meeting with Chairman Kim, he is not suggesting a meeting anytime soon, but is willing to meet if the need arises.\n\nOverall, President Lee emphasizes the importance of genuine cooperation, sincerity, and substantial results in both the setup of a liaison office and the possibility of a meeting with Chairman Kim.
gpt3.5_prediction	Question 1: Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue\n\nVerdict: 1.1 Explicit - The information requested is explicitly stated (in the requested form)\n\nExplanation: The response explicitly provides information about President Lee's suggestion to set up a permanent liaison office in Seoul and Pyongyang. It explains that the suggestion was made to facilitate dialogue and cooperation between North and South Korea and highlights the intention behind the suggestion.\n\nQuestion 2: Thoughts on proposing a meeting with Chairman Kim at an earlier date\n\nVerdict: 2.4 General - The information provided is too general/lacks the requested specificity.\n\nExplanation: The response does not specifically address the question of proposing a meeting with Chairman Kim at an earlier date. While President Lee expresses willingness to meet with Chairman Kim when the need is real and there can be substantial results, he does not provide a direct response to the request for thoughts on proposing an earlier meeting. The answer is more general in nature.
question	Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.
annotator_id	85
annotator1	None
annotator2	None
annotator3	None
inaudible	False
multiple_questions	False
affirmative_questions	True
index	2900
clarity_label	Ambivalent
evasion_label	Implicit

Figure 1: Sample

In addition, redundancy in the interview responses was observed. This may occur because some responses are similar despite referring to different questions, but mainly because the responses are labeled based on whether they address each sub-question. As a result, the same response may appear multiple times in the dataset, depending on the number of sub-questions associated with a particular interview question. This behavior is illustrated below by presenting the original question, the corresponding sub-questions, the interview response, and the assigned label. As shown, the same response is repeated across different sub-questions, while the assigned label may vary depending on the extent to which the response addresses each sub-question.

```
################################################################################################
INTERVIEW QUESTION:
Q. Mr. President, Turkey's ground offensive in northern Iraq is now a week old with no end in sight. How quickly would you like to see Turkey end its offensive, its incursion? And do you have any concerns abou
################################################################################################

--- Example ---
Sub-question:
How quickly would you like to see Turkey end its offensive, its incursion?

Answer:
A couple of points on that—one, the Turks, the Americans, and the Iraqis, including the Iraqi Kurds, share a common enemy in the PKK. And secondly, it's in nobody's interests that there be safe haven for peopl

Clarity label: Clear Reply

--- Example ---
Sub-question:
Do you have any concerns about the possibility of protracted presence in northern Iraq causing further destabilization in the region?

Answer:
A couple of points on that—one, the Turks, the Americans, and the Iraqis, including the Iraqi Kurds, share a common enemy in the PKK. And secondly, it's in nobody's interests that there be safe haven for peopl

Clarity label: Ambivalent
```

Figure 2: Sample

2.2.2. Descriptive Analysis.

Clarity Labels Distribution The labels that were used for the classification task are the clarity labels, which include Clear Reply, Clear Non-Reply, and Ambivalent. The distribution of instances across these labels is shown in Figure 3. The majority of instances belong to the Ambivalent class, accounting for approximately 59.2% of the data (2039 samples). The Clear Reply category represents about 30.5% of the dataset (1051 samples), while Clear Non-Reply is the least frequent class with 10.3% (356 samples). This distribution indicates a noticeable class imbalance, with the Ambivalent class dominating the dataset, a factor that should be considered during model training and evaluation.

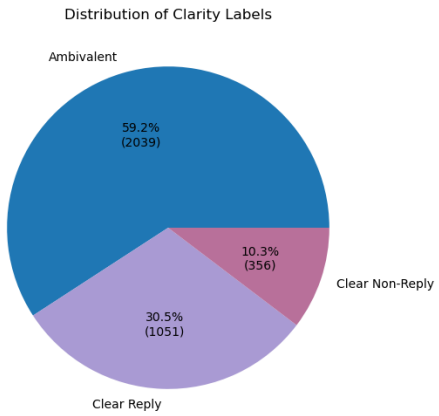


Figure 3: Distribution of clarity labels in the dataset

Evasion Labels Distribution The dataset also includes an evasion_label, which is a second, more detailed taxonomy that specifies the type of evasive strategy used in a response.

Figure 4 presents the relationship between clarity labels and evasion labels in the dataset. The evasion categories are Claims ignorance, Clarification, Declining to answer, Deflection, Dodging, Explicit, General, Implicit, and Partial/Half-answer. We can see that Deflection, Dodging, Implicit, General, and Partial/Half-answer correspond to Ambivalent responses, while Claims ignorance, Clarification, and Declining to answer correspond to Clear Non-Reply. The Explicit category corresponds to Clear Reply, meaning that the response is direct and clear.

evasion_label clarity_label	Claims ignorance	Clarification	Declining to answer	Deflection	Dodging	Explicit	General	Implicit	Partial/half-answer
Ambivalent	0	0	0	381	706	0	386	488	79
Clear Non-Reply	119	92	145	0	0	0	0	0	0
Clear Reply	0	0	0	0	0	1052	0	0	0

Figure 4: Relationship between clarity labels and evasion labels in the dataset.

Moreover, Figure 5 presents the distribution of the evasion labels in the dataset. The most frequent category is Explicit, accounting for 30.5% of the instances (1051 samples), which corresponds to clear and direct responses. Among the evasive strategies, Dodging appears most frequently (20.5%, 705 samples), followed by Implicit (14.2%, 487 samples), General (11.2%, 386 samples), and Deflection (11.0%, 380 samples). Less common categories include Declining to answer (4.2%), Claims ignorance (3.5%), Clarification (2.7%), and Partial/Half-answer (2.3%).

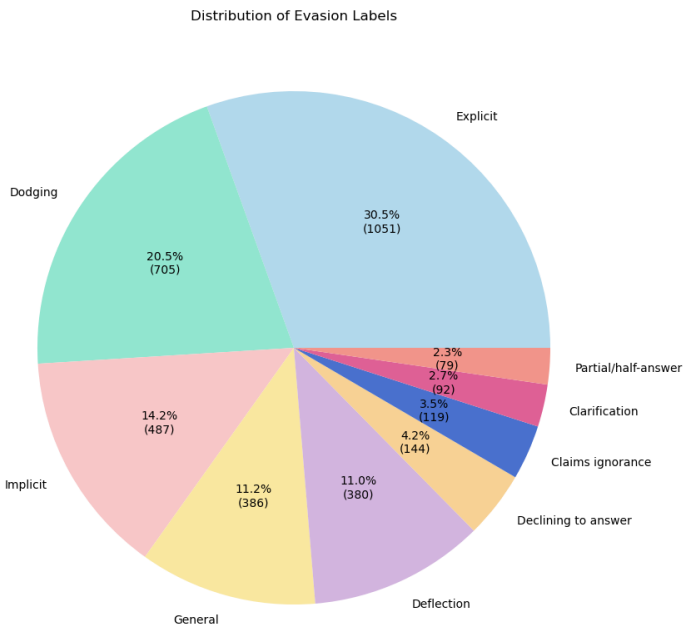
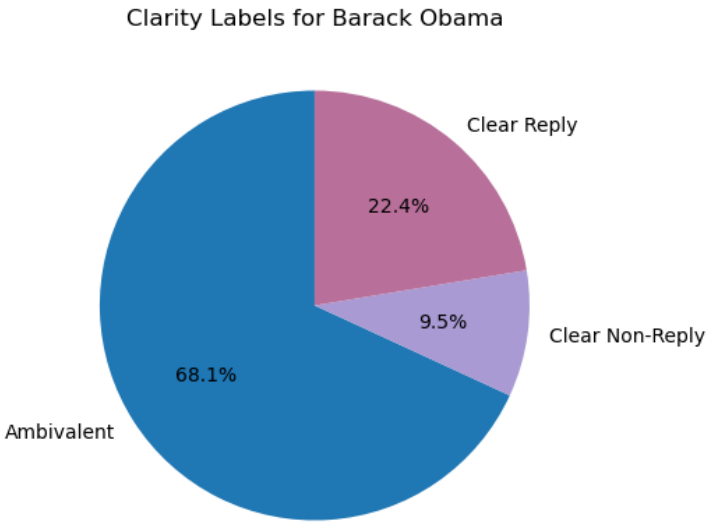
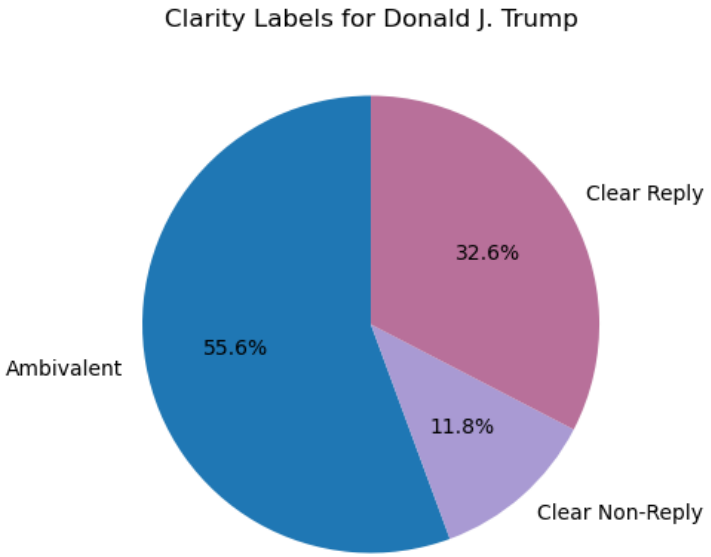


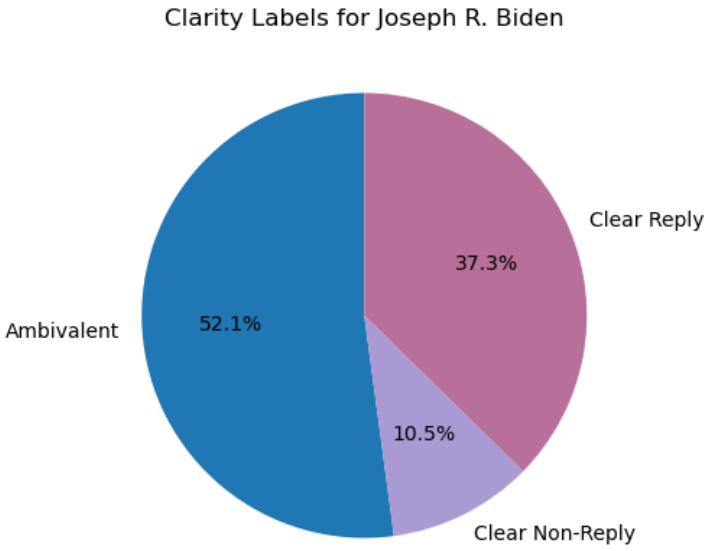
Figure 5: Distribution of evasion labels in the dataset.

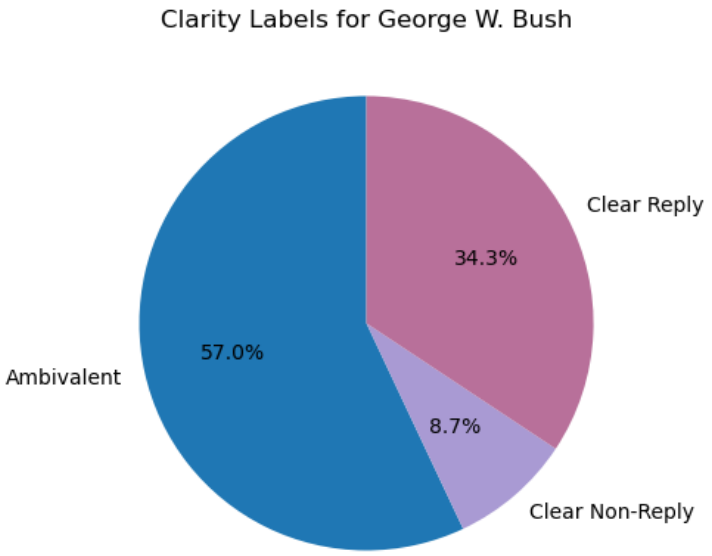
Clarity Label Distribution by President It is important to determine whether the nature of the responses in the dataset is speaker-dependent. An analysis of the distri-

bution of clarity labels for each president helps reveal whether certain categories tend to appear more frequently depending on the president who provides the response, rather than representing a general characteristic of the dataset as a whole. As illustrated in the figures, Ambivalent responses constitute the majority for all presidents. However, some differences can still be observed. In particular, Barack Obama exhibits the highest proportion of Ambivalent responses (68.1%), indicating a larger share of indirect or partially evasive answers. In contrast, Donald J. Trump shows a higher proportion of Clear Reply responses (32.6%), suggesting that he provides more direct answers compared to the other presidents.









To further compare these distributions, the label frequencies across presidents are visualized using a grouped bar chart. As shown in the figure, although minor differences can be observed, no substantial variations appear in the overall response patterns among the presidents. In all cases, Ambivalent responses remain the dominant category, followed by Clear Reply, while Clear Non-Reply appears less frequently. This suggests that the general response behavior is independent across the different presidents in the dataset.

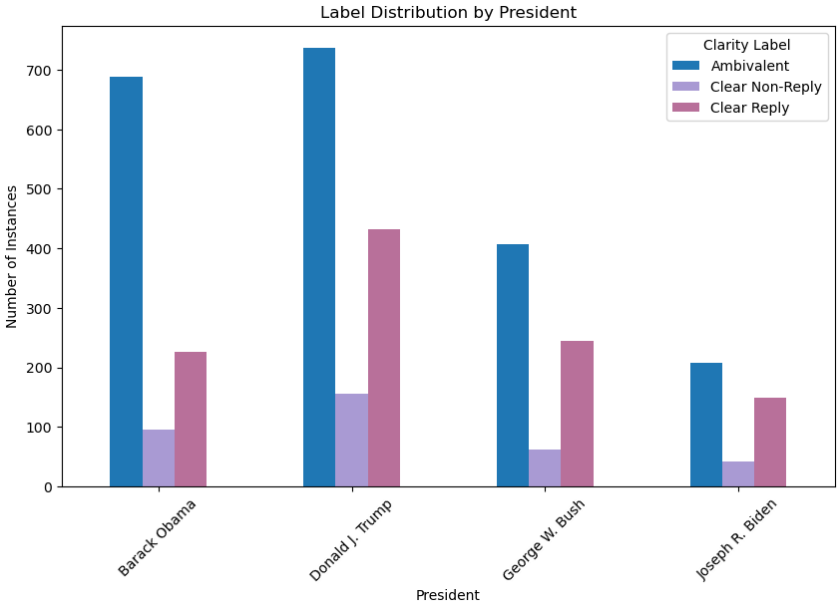


Figure 6: Label distribution per president

Clarity Labels Distribution by Affirmative questions Affirmative questions seem to be associated with slightly more direct responses. In these cases, Ambivalent responses become less frequent (from 60.5% to 54.5%), while Clear Reply (from 29.9% to 32.6%) and Clear Non-Reply responses (from 9.6% to 12.8%) become slightly more common. This suggests that affirmative questions may encourage clearer answers, although the effect remains limited.

clarity_label	Ambivalent	Clear Non-Reply	Clear Reply
affirmative_questions			
False	0.605007	0.096039	0.298954
True	0.545337	0.128238	0.326425

Figure 7: Label distribution per affirmative questions

Clarity Labels Distribution by Year Figure 8 shows a heatmap of interview questions by president and year. The distribution of questions matches the years of each presidency, with questions concentrated in the relevant time periods. Some differences can also be seen in the number of questions per year, since some years contain more instances than others.

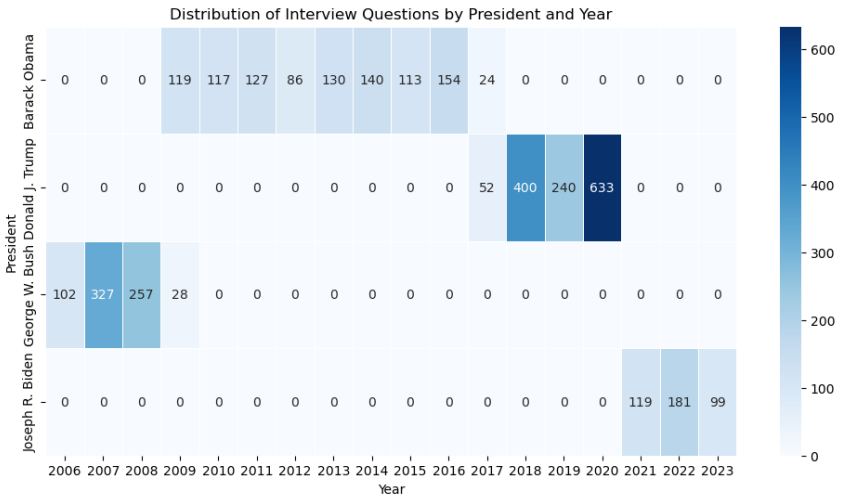


Figure 8: Distribution of interview answers per presidents and year

Figure 9 illustrates the distribution of clarity labels across different years. As observed, there is no significant temporal variation in the distribution, as the relative proportions of the labels remain stable over time.

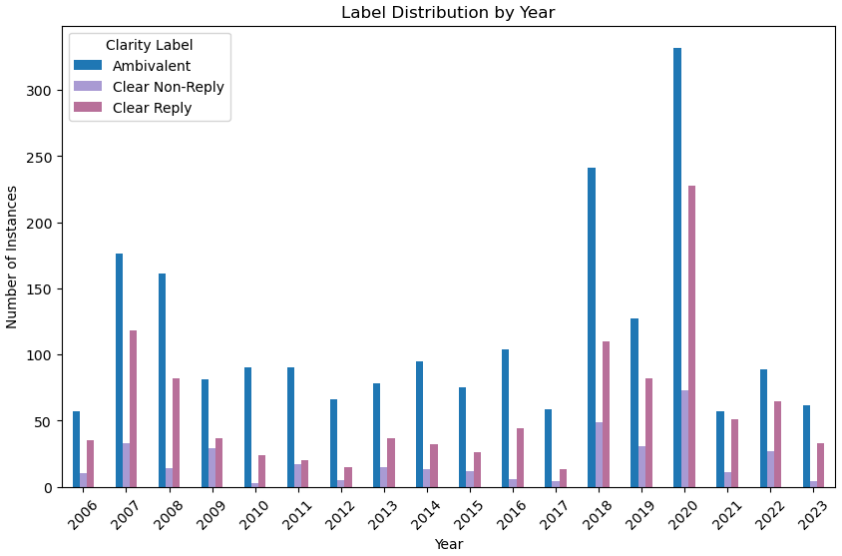


Figure 9: Distribution of clarity labels by year

2.2.3. Textual Analysis. The analysis of the dataset then focused on its textual characteristics. This type of analysis helps us better understand the linguistic properties of the responses, such as their length, vocabulary, and recurring word patterns. It also helps show whether certain textual features are related to specific clarity labels and whether the classes can be distinguished based on simple language patterns. In addition, this analysis can reveal possible challenges for the classification task, such as similar vocabulary across classes, the lack of clear lexical cues, or patterns that need preprocessing. These findings support the use of more informative text representations in the next stage of modeling.

Clarity Label Distribution by Word Count The analysis of answer length, measured in number of words, reveals notable differences across the clarity labels. The dataset contains 3,448 responses, with an average length of approximately 294 words and a high standard deviation (301.54), which indicates a variability in response length. The distribution is highly skewed, ranging from very short answers (1 word) to very long ones (up to 2,117 words). The median value is lower than the mean, suggesting the presence of long outlier responses that increase the average,.

When examining the classes separately, Ambivalent responses not only constitute the majority of the dataset but also tend to be longer on average (331.86 words). Clear Reply responses include 1,052 samples with a moderate average length of 272.04 words. In contrast, Clear Non-Reply responses form the smallest group and are considerably shorter, with an average length of 137.81 words.

Figure 10 illustrates these differences in answer length across the clarity labels.The boxplot shows that responses labeled as Ambivalent and Clear Reply tend to contain more words and exhibit greater variability in length compared to Clear Non-Reply. In contrast, Clear Non-Reply responses are generally shorter, with a lower median word count and a more concentrated distribution, although all classes contain several outliers.

Overall, these results suggest a clear relationship between response length and clarity labels. Shorter responses are more frequently associated with non-replies, while longer and more variable responses tend to correspond to ambivalent answers. This observation suggests that response length may serve as a useful feature for distinguishing between clarity categories.

```
=== Overall statistics ===
count    3448.000000
mean      293.572216
std       301.541101
min         1.000000
25%        56.000000
50%       207.000000
75%       440.000000
max      2117.000000
Name: answer_word_count, dtype: float64
Smallest answer: 1
Largest answer: 2117

=== Statistics by clarity_label ===
                count        mean  median  min   max         std
clarity_label
Ambivalent       2040  331.857843   243.5    1  2117  319.526222
Clear Non-Reply   356  137.806180    39.5    1  1407  217.389827
Clear Reply      1052  272.041825   194.0    1  1592  269.540724

<Figure size 800x500 with 0 Axes>
```

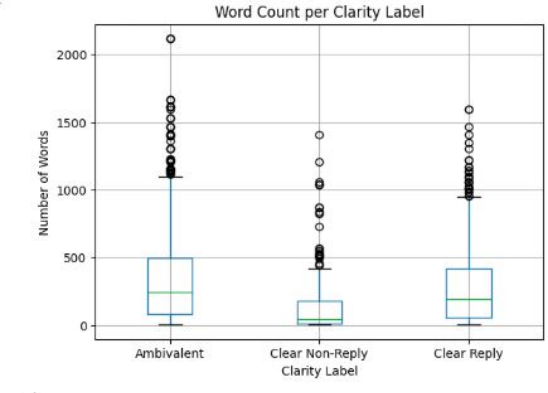


Figure 10: Words per label

Clarity Label Analysis Based on Unigrams, Bigrams, and Trigrams The most frequent unigrams, bigrams, and trigrams were computed across the corpus after simple cleaning steps, such as removing punctuation, in order to examine whether specific lexical patterns could help distinguish between the different clarity labels. Based on the tables, the first clear observation is that there is substantial lexical overlap across the three classes. For example going, think, people, know, president, want, states, and united appear in more than one category. More over united states, american people, prime minister, making sure, and north korea are also shared across labels. The same pattern appears in the trigrams. This suggests that the classes are not separated by topic, or phrases alone, since similar political themes and general expressions are present throughout the dataset. At the same time, some differences between the categories can still be observed. The Ambivalent class includes broad political expressions such as international community, making sure, and find common ground, which suggest more general or less committed answers. The Clear Non-Reply category contains phrases such as going comment, thank thank, and answer little bit, which may reflect hesitation or avoidance. In contrast, the Clear Reply category includes more specific expressions, such as wall street journal, new york times, and use chemical weapons, suggesting more explicit and content-focused responses.

The Clear Non-Reply category seems to contain fewer clear topic-related phrases and more fragmented expressions. For example, trigrams such as different time looking lot and things including cabinet happy do not point to a specific topic, but instead reflect less coherent and more locally structured language. This suggests that these responses may be characterized more by evasive wording than by specific content. In contrast, the Clear Reply and Ambivalent categories appear to contain more coherent topical phrases, such as use chemical weapons, kim jong un, or international community.

Overall, the tables suggest that simple n-gram frequency analysis reveals some weak category-level tendencies, but not a strong lexical boundary between the labels. The overlap between the most frequent words and phrases is too large for these features alone to clearly separate the classes.

Table 2: Top 15 unigrams by clarity label

Rank	Ambivalent		Clear Non-Reply		Clear Reply	
1	going	(4218)	going	(357)	think	(1796)
2	think	(3946)	think	(276)	people	(1718)
3	people	(3627)	know	(272)	going	(1677)
4	know	(1976)	people	(248)	know	(971)
5	president	(1932)	like	(143)	president	(794)
6	states	(1565)	want	(139)	want	(670)
7	like	(1562)	question	(125)	like	(653)
8	want	(1555)	president	(119)	way	(637)
9	united	(1528)	lot	(117)	said	(633)
10	said	(1477)	time	(109)	states	(585)
11	country	(1369)	sure	(97)	united	(584)
12	way	(1361)	united	(97)	time	(563)
13	time	(1280)	right	(97)	world	(544)
14	lot	(1271)	states	(91)	lot	(534)
15	world	(1229)	work	(90)	country	(525)

Table 3: Top 15 bigrams by clarity label

Rank	Ambivalent		Clear Non-Reply		Clear Reply	
1	united states	(1378)	united states	(85)	united states	(527)
2	american people	(418)	north korea	(44)	american people	(181)
3	prime minister	(345)	prime minister	(29)	prime minister	(126)
4	making sure	(249)	american people	(26)	think going	(117)
5	think going	(249)	making sure	(23)	north korea	(111)
6	north korea	(218)	long time	(21)	lot people	(90)
7	international community	(186)	lot people	(19)	making sure	(82)
8	long term	(174)	little bit	(18)	international community	(74)
9	mr president	(165)	national security	(17)	health care	(72)
10	health care	(165)	new york	(16)	white house	(61)
11	think important	(165)	know going	(15)	think important	(59)
12	lot people	(152)	going comment	(15)	long term	(58)
13	long time	(149)	thank thank	(13)	mr president	(57)
14	middle east	(138)	international community	(13)	long time	(54)
15	right thing	(132)	going continue	(12)	going continue	(52)

Table 4: Top 15 trigrams by clarity label

Rank	Ambivalent	Clear Non-Reply	Clear Reply
1	president united states	(111) united states america	(8) united states america (35)
2	united states america	(91) asia pacific region	(8) president united states (33)
3	free trade agreement	(55) jobs united states	(8) wall street journal (27)
4	think american people	(41) low cost labor	(8) free trade agreement (26)
5	middle class families	(41) lot different things	(7) new york times (23)
6	new york times	(41) soon long time	(6) middle class families (16)
7	prime minister netanyahu	(39) answer little bit	(6) use chemical weapons (16)
8	wall street journal	(38) little bit different	(6) united states going (14)
9	find common ground	(37) bit different time	(6) john mckinnon wall (14)
10	united states going	(31) different time looking	(6) mckinnon wall street (14)
11	long period time	(29) time looking lot	(6) china united states (13)
12	kaesong industrial complex	(28) looking lot different	(6) american people want (13)
13	security council resolution	(27) different things including	(6) kim jong un (13)
14	kim jong un	(26) things including cabinet	(6) find common ground (13)
15	spent lot time	(26) including cabinet happy	(6) trans pacific partnership (12)

Detection of text paterns in the dataset Although it is possible to manually inspect several examples from the dataset (e.g., by printing random entries), such inspection does not guarantee that all instances follow the same format. To systematically verify whether patterns are present in the dataset, a function named detect_text_patterns was implemented that scans the textual data and reports the occurrence of common types of noise. Specifically, the function searches for patterns such as HTML tags, email addresses, URLs, Twitter mentions, hashtags, digits, and newline characters using regular expressions. Additionally, it checks for the presence of emojis in the text. This automated inspection provides a quick overview of the dataset and helps determine whether specific cleaning operations are required before further processing. The results of this analysis indicated that the only notable pattern detected in the text column was the presence of numerical values.

```
detect_text_patterns(training_data["text"])
✓ 6.6s

Starting dataset inspection...
----------------------------------------
Clean: No HTML Tags found.

Clean: No Emails found.

Clean: No URLs (http/www) found.

Clean: No Twitter Mentions (@) found.

Clean: No Hashtags (#) found.

Found: 1527 texts with Digits.
Clean: No Newlines (\n) found.

Clean: No Emojis found.

----------------------------------------
Inspection completed!
```

Figure 11: Detection Patterns

Detection of spelling errors Moreover, a quality check was conducted to estimate the percentage of spelling errors present in the dataset. To do so, a function was implemented that scans the textual data and compares each word against an English dictionary using a spell-checking library. The function tokenizes the text into individual words, counts the total number of words, and identifies those that are not recognized by the dictionary as potential spelling errors. It then computes the percentage of misspelled words relative to the total number of words in the dataset. In addition, the function outputs a small set of example misspelled words to provide a qualitative indication of the types of spelling inconsistencies present in the data.

As shown in Figure 12, the analysis indicates that spelling errors constitute a very small portion of the dataset (approximately 1.96% of all words), suggesting that the textual data is generally clean and that spelling mistakes are unlikely to significantly affect the performance of the classification models. After removing stopwords, which usually carry limited semantic information for classification tasks, the percentage of misspelled words increased slightly to 2.94%. This suggests that spelling errors are somewhat more common in the more informative parts of the text, although their overall proportion remains relatively low.

```
check_spelling_errors(training_data["text"])
✓ 4.4s

Total words: 1125356
Misspelled words: 22040
Percentage of misspelled words: 1.96%
Example misspelled words: ['thei', 'chineseno', 'aboutit', 've', 'donei']
```

Figure 12: Spelling errors

3. Pre-processing

3.1. The Working dataset

As a first step, the working dataset used in the experiments was created by selecting the most relevant columns from the original training data. In particular, the processed sub-question, the corresponding interview answer, and the target label (clarity_label) were kept, since they contain the core information needed for the clarity classification task.

However, as mentioned earlier, the same interview response may appear multiple times in the dataset, each time paired with different sub-questions and, therefore, different labels.This can be misleading for the model, since identical responses may correspond to different target classes. To address this issue and better capture the relationship between sub-questions, responses, and their corresponding labels, the dataset was enriched with additional features.

Specifically, a new textual field named text was created by concatenating the sub-question and the corresponding interview answer into a single representation. This allows the model to jointly consider the question context and the response, which is crucial for accurately assessing clarity.For the Word2Vec approach in particular, this concatenated field was formatted using the prefixes "Question:" and "Answer:", so that the combined text explicitly preserved the distinction between the two parts.

Furthermore, an additional feature representing the length of the text, measured in number of words, was introduced for each sub-question–response pair. During the exploratory analysis, a relationship was observed between response length and the clarity labels, with shorter responses more often associated with non-replies and longer ones with ambivalent answers. Therefore, incorporating this feature may provide the model with additional discriminative information and potentially improve its ability to distinguish between classes.

Figure 13.

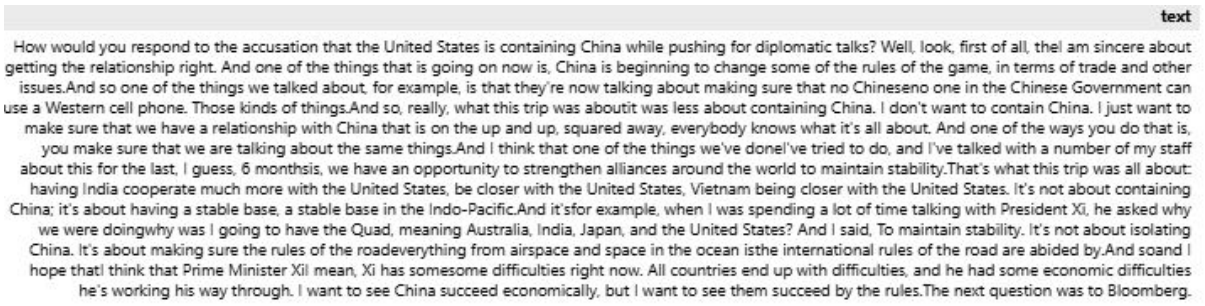


Figure 13: "Text" column sample

Text cleaning After identifying the types of noise that may appear in the dataset, we designed a general and configurable text cleaning function named clean_text. The purpose of this function is to provide a flexible preprocessing pipeline in which different cleaning operations can be enabled or disabled, allowing us to experiment with different preprocessing configurations during the experiments.

The function supports several common preprocessing steps, including lowercasing, contraction expansion, removal of URLs, numbers, and non-alphabetic characters, normalization of whitespace, and reduction of repeated characters. When token-level processing is required, the function can also remove short words based on a minimum word length, filter stopwords either using a predefined list or a custom one, apply lemmatization using spaCy, perform stemming with the Porter stemmer, and optionally remove duplicate tokens.

This modular design allows us to easily modify the preprocessing configuration across different experiments and evaluate how different combinations of cleaning steps affect the performance of the classification models.

Data partitioning for train, test and validation The dataset provides the training and test sets separately. However, relying on a single train–test–validation split is not enough to evaluate the classification models reliably.

For this reason, stratified k-fold cross-validation was used. This method was chosen because the dataset is imbalanced, and stratification ensures that each fold preserves the same class distribution as the original dataset. As a result, the evaluation of model performance is more stable and reliable.

3.2. Vectorization

Before applying a machine learning model such as Logistic Regression, the textual data must first be transformed into a numerical representation. Since classification algorithms cannot directly process raw text, each document needs to be converted into a vector of numerical features.

3.2.1. TF-IDF. The first vectorization method examined in this study is Term Frequency–Inverse Document Frequency (TF-IDF). TF-IDF assigns a numerical weight to each word based on two factors: its frequency within a specific document and its rarity across the entire corpus. Specifically, TF-IDF is calculated as the product of two terms: Term Frequency (TF) and Inverse Document Frequency (IDF). The IDF component is calculated as:

$$\mathrm{IDF}(t) = \log\left(\frac{N}{df(t)}\right)$$

where N is the total number of documents in the corpus and $df(t)$ is the number of documents containing the term t. This formulation assigns higher weights to terms that are frequent in a specific document but rare across the corpus. As a result, TF-IDF highlights words that are more informative and useful for distinguishing between classes.

To implement this approach, a custom class named TFIDFProcessor was developed. This class is responsible for converting textual data into TF-IDF feature representations using the TfidfVectorizer provided by the scikit-learn library. More specifically, the fit_transform() method is applied to the training texts in order to learn the vocabulary and generate the corresponding sparse TF-IDF matrix, while the transform() method is used to convert new texts, such as the test set, based on the already fitted vocabulary. In addition, the class stores the extracted feature names and provides a summary of

the generated feature matrix, including the vocabulary size and the dimensionality of the representation.

Beyond standard vectorization, a second custom class named TFIDFExplorer was implemented to support further analysis of the extracted TF-IDF features. This class includes diagnostic functions for exploring the generated vocabulary and inspecting word importance both globally and within each target class. More specifically, it can display randomly selected words from the vocabulary, identify the top words with the highest mean TF-IDF scores across the entire dataset, and compute the most important words separately for each class label. These functions were designed to examine whether specific terms are more strongly associated with particular classes, thereby providing deeper insight into the structure of the feature space.

3.2.2. Word2Vec. Representing words solely based on their frequency is limited because it ignores context and meaning. Since words can have multiple meanings depending on their usage, simple frequency-based representations are often inadequate for more complex natural language processing tasks.

To address this limitation, Word2Vec is used to generate word embeddings, transforming each word into a dense numerical vector. In this vector space, words that appear in similar contexts are placed closer to each other, with similarity typically measured using cosine similarity. Word2Vec is a neural network-based method for learning such embeddings and can be trained using two main architectures: Continuous Bag-of-Words (CBOW) and Skip-gram. CBOW predicts a target word from its surrounding context, whereas Skip-gram predicts the surrounding context words given a target word. In general, CBOW is faster and performs well for frequent words, while Skip-gram is often more effective at capturing semantic relationships involving less frequent words.

In this study, the Word2Vec-based representation is implemented through a class-based design. More specifically, a base class named BaseW2VAnalyzer is used to handle the common functionality required for text processing and document vector construction. The supported pooling strategies are mean, max, mean_max, and TF-IDF weighted mean. Since classification models such as Logistic Regression require a single fixed-size numerical representation for each document, the individual word embeddings must be aggregated into one document-level vector. In the mean approach, the document representation is obtained by averaging the embeddings of all valid words in the document. In the max approach, the maximum value across each embedding dimension is selected. In the mean_max approach, the mean and max vectors are concatenated into a single representation. In the TF-IDF weighted mean approach, each word vector is weighted according to its TF-IDF importance before averaging, so that more informative words contribute more strongly to the final document representation.

Two different subclasses are then used to support the two Word2Vec approaches examined in this work. The first one, PretrainedWord2VecAnalyzer, is responsible for loading an already trained Word2Vec model. This makes it possible to use pre-trained embeddings, such as those learned from large external corpora, without retraining the model on the current dataset. The second one, Word2VecAnalyzer, is used to train a new Word2Vec model directly on the dataset of the study. This class allows the configuration of important training parameters such as vector dimensionality, context window size, minimum word frequency, number of worker threads, training epochs, and the

choice between Skip-gram and CBOW through the sg parameter.

4. Model

4.1. Logistic regression

Logistic Regression is a supervised machine learning algorithm commonly used for classification tasks. Despite its name, it is a linear model designed to estimate the probability that a given input belongs to a particular class. It achieves this by applying the logistic (sigmoid) function to a linear combination of the input features.

4.2. Hyper-parameter tuning

To optimize the performance of the Logistic Regression model, a hyperparameter tuning process was carried out using Grid Search combined with stratified cross-validation. Several values were tested for the main hyperparameters, including the regularization strength C, the type of regularization ($L1$ or $L2$), the solver used for optimization (liblinear, saga, or lbfgs), and the use of class balancing through the class_weight parameter. For each hyperparameter combination, the model was trained and evaluated using Stratified K-Fold Cross-Validation, with shuffling enabled and a fixed random state for reproducibility. This ensured that the class distribution remained consistent across all folds. Model selection was based on the macro-averaged F1-score (f1_macro), which is particularly appropriate for imbalanced datasets because it gives equal importance to the predictive performance of all classes.

Finally, the hyperparameter combination that achieved the highest mean macro F1-score across the cross-validation folds was selected as the best configuration. The corresponding best estimator was then returned and used as the final Logistic Regression model, with the aim of improving generalization on unseen data.

4.3. Evaluation

To evaluate the performance of the Logistic Regression model, a comprehensive evaluation framework was implemented through the `ModelEvaluator` class. This framework computes a range of standard classification metrics, including Accuracy, Precision, Recall, and F1-score. More specifically, the implemented evaluation reports both macro-averaged and weighted versions of Precision, Recall, and F1-score, allowing a more detailed assessment of model behavior across classes.

Due to class imbalance in the dataset, the analysis primarily focuses on the F1-score, which represents the harmonic mean of Precision and Recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1}$$

For a more complete assessment of the model, two variations of the F1-score are considered:

- **Macro F1-score**: the unweighted average of the F1-scores across all classes. It treats all classes equally and is especially informative when the dataset is imbalanced, since it reflects performance on minority classes as well.

- **Weighted F1-score**: the average of the F1-scores weighted by the number of samples in each class. This metric reflects performance while taking the true class distribution into account.

The evaluation process is based on **Stratified K-Fold Cross-Validation**, with shuffling enabled and a fixed `random_state` to ensure reproducibility. This strategy preserves the class distribution across folds and provides a more reliable estimate of model performance on unseen data. During cross-validation, the framework computes the mean and standard deviation of all selected metrics across folds, offering both a central performance estimate and an indication of variability.

In addition to metric computation, the `ModelEvaluator` class generates cross-validated predictions using `cross_val_predict`. These predictions are then used to produce a full classification report as well as a confusion matrix, allowing a more detailed inspection of class-wise performance and misclassification patterns.

Finally, the framework includes a learning curve based on the macro F1-score. This visualization compares training and validation performance over increasing training set sizes and helps diagnose model behavior:

- **High Variance (Overfitting)**: a large gap between training and validation performance suggests that the model may not generalize well and may benefit from stronger regularization or additional training data.
- **High Bias (Underfitting)**: low performance on both curves suggests that the model or the selected feature representation is not sufficiently expressive to capture the complexity of the classification task.

Overall, this evaluation framework provides both quantitative and visual tools for assessing the robustness, generalization ability, and class-wise behavior of the Logistic Regression model.

5. Experiments

To ensure a consistent experimental process, a text processing pipeline was implemented for all experiments. This pipeline includes preprocessing, feature extraction, model training, and evaluation.

To manage this process, a dedicated class named ExperimentTracker was developed. This class organizes the full workflow, including text cleaning, vectorization, model training with hyperparameter tuning, cross-validation, and final evaluation on the test set. By using this structure, all experiments are executed in a consistent and reproducible way.

5.1. Experiments for TF-IDF Vectorization

Baseline TF-IDF The baseline TF-IDF model consists of a text classification pipeline that applies basic preprocessing, TF-IDF feature extraction, and Logistic Regression for classification. The input text is cleaned by converting all characters to lowercase, removing non-alphabetic characters, and eliminating numerical values.

The cleaned text is then transformed into a numerical representation using TF-IDF. In the baseline setting, the vectorizer uses its default configuration, which corresponds to a unigram-based representation without an explicit vocabulary-size restriction and without stopword removal. The vectorizer applies inverse document frequency weighting and L2 normalization, resulting in sparse document vectors.

For classification, Logistic Regression is used as the classifier, and its hyperparameters are optimized through Grid Search with 5-fold Stratified Cross-Validation. Model selection is based on the macro F1-score in order to account for the class imbalance of the dataset.

This baseline serves as the reference point for the subsequent TF-IDF experiments. Overall, the baseline shows that the model performs more reliably on the majority class, while the two non-majority classes remain more difficult to distinguish. The confusion matrix reflects this behavior, and the learning curves suggest limited generalization, with a gap between training and validation performance.

Preprocessing Techniques for TF-IDF A second group of experiments examined whether preprocessing choices could improve the baseline TF-IDF representation. In particular, stemming, lemmatization, and different stopword-removal strategies were explored.

For stemming and lemmatization, the baseline cleaning pipeline was extended with the corresponding normalization step, while the TF-IDF representation and classification model remained unchanged. These experiments were designed to test whether reducing inflectional variation could improve the consistency of the textual representation.

In addition, stopword removal was tested using both a default stopword list and a custom stopword list. The custom stopwords were selected from highly frequent terms that were expected to contribute limited discriminative information. The goal was to reduce noise while preserving words that are more relevant to class separation.

Overall, these preprocessing variations led only to modest changes in performance. Linguistic normalization provided slight improvements over the baseline, and stopword removal was somewhat more helpful when a custom list was used. However, the general class-wise behavior of the model remained largely unchanged, with the majority class still being easier to classify than the others.

TF-IDF Parameter Tuning After selecting the most effective preprocessing pipeline, further experiments focused on the configuration of the TF-IDF vectorizer itself. These experiments explored the effect of adding higher-order n-grams, restricting the feature space, and adjusting document-frequency thresholds.

First, the feature representation was extended to include unigrams and bigrams, in order to capture short contextual patterns in addition to individual words. This configuration improved the representation compared with simpler unigram-based settings, suggesting that local phrase information is useful for the task.

Next, the n-gram range was further extended to include trigrams. Although this produced a richer representation, it did not lead to a clear improvement over the unigram-bigram setting. This indicates that adding longer n-grams increases representational complexity, but the additional context is not necessarily translated into better classification performance.

Finally, a more constrained TF-IDF configuration was tested by combining unigram-bigram features with a vocabulary-size limit, frequency thresholds, and sublinear term-frequency scaling. This setup yielded the strongest TF-IDF-only performance in the notebook, indicating that a more selective and regularized feature space improves the quality of the representation. Overall, the parameter-tuning experiments show that TF-IDF benefits more from careful vectorizer design than from preprocessing changes alone.

Final TF-IDF Experiment In the final TF-IDF experiment, textual features were combined with an additional numerical feature corresponding to the answer word count. The text was preprocessed using the best-performing TF-IDF cleaning pipeline, while the numerical feature was standardized separately.

A ColumnTransformer was used to combine the TF-IDF representation of the text with the scaled numerical feature into a single feature space. Logistic Regression was then trained on this combined representation, again using Grid Search with 5-fold Stratified Cross-Validation for hyperparameter selection.

This final configuration produced performance very close to the best text-only TF-IDF setting. Therefore, the answer word count appears to provide only limited additional discriminative information beyond the textual features. The overall behavior of the model remained similar, and the learning curves continued to suggest that overfitting was still present.

Summary of Results Overall, the TF-IDF experiments show that the model can be improved incrementally through preprocessing and vectorizer tuning, but the gains remain relatively limited. The most meaningful improvements were obtained through vectorizer parameter tuning, especially by combining unigram-bigram features with vocabulary and document-frequency constraints.

At the same time, the experiments consistently show that the model classifies the majority class more accurately than the other two classes. This pattern is also reflected in the confusion matrices, where the *Ambivalent* class is classified more successfully, while *Clear Reply* and especially *Clear Non-Reply* are more often confused with other categories.

In addition, the learning curves across the TF-IDF experiments consistently indicate overfitting. In most cases, a noticeable gap remains between training and validation performance, showing that the model learns the training data more effectively than it generalizes to unseen samples. Therefore, the main limitation of this family of models appears to be not only the choice of features, but also the restricted generalization ability of the classifier. A representative example of this behavior is presented in the corresponding figure.

Figure 13.

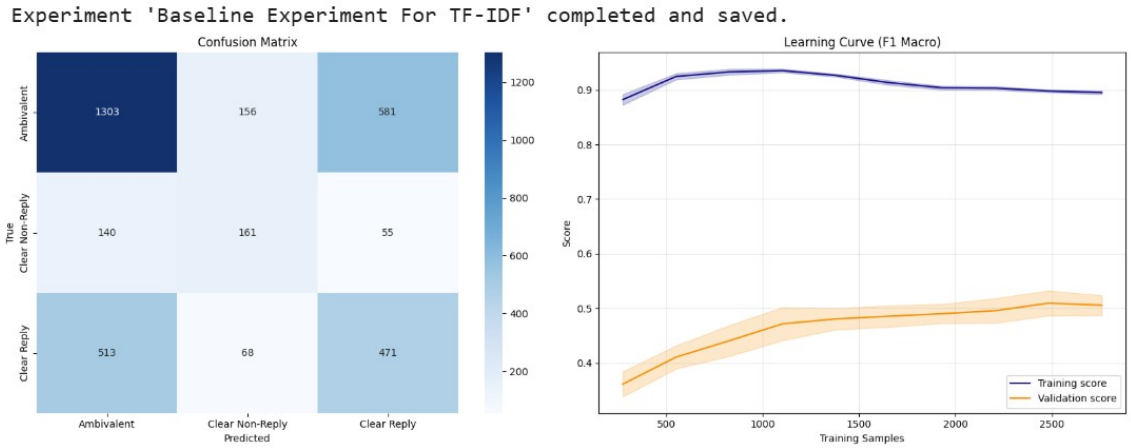


Figure 14: Sample of confusion matrix and learning curve

5.2. Experiments for Word2Vec Representations

Before conducting the Word2Vec experiments, the text representation used in the dataset was modified in order to provide richer contextual information. Instead of using only the interview answer, a new textual field was created by concatenating the interview question and the corresponding answer into a single sequence. Each sample was therefore represented in the form "Question: [question] Answer: [interview_answer]". This representation was used throughout all Word2Vec-based experiments.

Baseline Word2Vec The baseline Word2Vec experiment used a simple preprocessing configuration in which the text was only converted to lowercase. No further cleaning, stopword removal, stemming, or lemmatization was applied, in order to preserve as much lexical information as possible for the embedding model.

For text representation, a Word2Vec model was trained directly on the corpus of the training set. The model used fixed-dimensional word embeddings, a small context window, and mean pooling in order to derive a single vector representation for each document from its word embeddings.

These document representations were then used as input to a Logistic Regression classifier, whose hyperparameters were optimized using Grid Search. The same evaluation protocol as in the TF-IDF experiments was retained, namely 5-fold Stratified Cross-Validation with macro F1-score as the model-selection criterion.

This baseline serves as the reference point for all subsequent Word2Vec experiments. Compared with the TF-IDF experiments, the Word2Vec baseline shows a somewhat more stable training-validation behavior, but it still struggles with the same class-separation problem, especially for the two non-majority classes.

Pretrained Word2Vec Baseline A second baseline experiment was conducted using pretrained Word2Vec embeddings from the Google News model. The same text construction strategy and lowercasing step were retained, while the word representations were obtained from an external pretrained model rather than being learned from the task-specific corpus.

As in the baseline experiment, mean pooling was used to obtain document-level vectors, and these vectors were used to train a Logistic Regression classifier with hyperparameter optimization through Grid Search.

In the notebook, this pretrained variant did not outperform the Word2Vec model trained directly on the dataset. This suggests that the external pretrained embeddings, although semantically rich, are not as well aligned with the characteristics of the specific classification task as embeddings learned from the task-specific corpus.

Preprocessing Variations for Word2Vec Additional Word2Vec experiments investigated the effect of stopword removal and lemmatization. Stopword removal was tested using both a custom stopword list and a default stopword-removal setting, while keeping the rest of the Word2Vec configuration unchanged.

The results indicate that stopword removal does not improve the Word2Vec representation in this task. This is plausible, since Word2Vec depends on distributional context, and removing frequent function words may reduce useful contextual structure rather than only removing noise.

Lemmatization, by contrast, produced a representation that was slightly more competitive than the baseline and served as the basis for the subsequent parameter-tuning experiments. This suggests that some degree of lexical normalization may help stabilize the learned embeddings, although the effect remains limited.

Word2Vec Parameter Experiments After retaining lemmatization as the most promising preprocessing choice, a series of experiments examined the effect of modifying the Word2Vec configuration itself.

First, the embedding dimensionality was increased in order to test whether a larger semantic space could improve document representations. This change did not lead to a substantial improvement, indicating that the baseline dimensionality was already sufficient for the task.

Next, the Skip-Gram variant was tested in place of the default training strategy. In the notebook, this setting did not improve the results, suggesting that the alternative training objective is less suitable in this particular setup.

Different pooling strategies were also explored, including max pooling and TF-IDF-weighted mean pooling. Neither of these alternatives outperformed simple mean pooling, which remained the most effective aggregation strategy for converting word embeddings into document-level vectors.

Finally, a larger context window was tested. This adjustment also failed to produce a clear improvement, suggesting that broader local context does not necessarily provide more useful information for this task.

Combined Word2Vec Configuration The final Word2Vec configuration combined the most promising settings identified in the earlier experiments. In the notebook, this combined model used lemmatization together with an increased embedding size, while retaining mean pooling and a small context window.

This combined setup produced the strongest Word2Vec result among the tested Word2Vec variants, although the improvement over the simpler Word2Vec configurations remained modest. The overall pattern of predictions was still similar to that observed in the earlier experiments: the majority class was easier to classify, whereas the two remaining classes continued to be more difficult to separate.

Summary of Results Overall, the Word2Vec experiments show that this family of representations does not substantially outperform the TF-IDF-based models in this task. Most of the tested changes resulted in only small differences, and the general classification behavior remained stable across experiments.

The most competitive Word2Vec setups were obtained with lemmatization and the combined configuration that increased the embedding dimensionality while preserving mean pooling. At the same time, stopword removal, Skip-Gram training, alternative pooling methods, and larger context windows did not provide meaningful benefits.

An important qualitative difference from TF-IDF is that the learning curves suggest less pronounced overfitting. Nevertheless, this more stable training behavior does not translate into clearly superior classification performance. The main challenge therefore remains the limited separability of the classes, rather than only the choice of embedding model.

Figure 13.

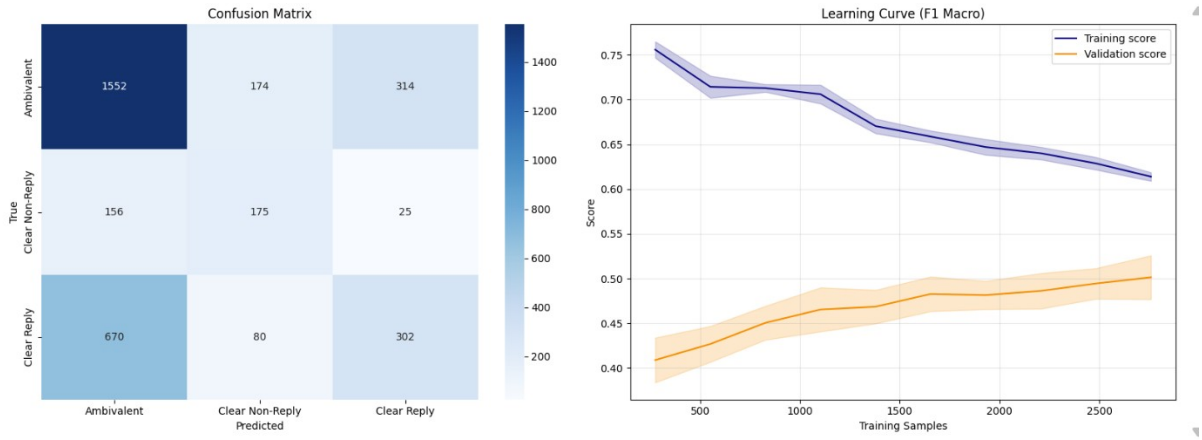


Figure 15: Sample of confusion matrix and learning curve

5.3. Results Analysis

Table 5: Performance of the TF-IDF-based experiments

Experiment	F1-macro	F1-weighted
Baseline	0.5064 ± 0.0188	0.5634 ± 0.0101
Stemming	0.5100 ± 0.0067	0.5570 ± 0.0102
Lemmatization	0.5107 ± 0.0192	0.5566 ± 0.0195
Default stop words	0.5115 ± 0.0137	0.5595 ± 0.0085
Custom stop words	0.5146 ± 0.0139	0.5847 ± 0.0097
TF-IDF with unigrams + bigrams	0.5300 ± 0.0108	0.5806 ± 0.0076
TF-IDF with unigrams + bigrams + trigrams	0.5284 ± 0.0143	0.5788 ± 0.0114
TF-IDF with tuned parameters	0.5330 ± 0.0179	0.5975 ± 0.0109
TF-IDF + answer_word_count	0.5332 ± 0.0122	0.5965 ± 0.0094

Table 6: Performance of the Word2Vec-based experiments

Experiment	**F1-macro**	**F1-weighted**
Custom-trained Word2Vec baseline	0.4964 ± 0.0168	0.5698 ± 0.0188
Pretrained Word2Vec (Google News)	0.4648 ± 0.0205	0.5529 ± 0.0166
Default stop words	0.4759 ± 0.0178	0.5575 ± 0.0142
Custom stop words	0.4953 ± 0.0072	0.5694 ± 0.0081
Lemmatization	0.5019 ± 0.0225	0.5703 ± 0.0228
Vector size = 500	0.4997 ± 0.0123	0.5707 ± 0.0149
Skip-Gram ($sg = 1$)	0.4896 ± 0.0099	0.5474 ± 0.0142
Max pooling	0.4551 ± 0.0124	0.5479 ± 0.0081
TF-IDF weighted mean pooling	0.4828 ± 0.0092	0.5568 ± 0.0126
Increased window size	0.4948 ± 0.0205	0.5685 ± 0.0156
Combined model	0.5009 ± 0.0233	0.5700 ± 0.0195

5.3.1. Results.

5.3.2. Best trial. Overall, the Word2Vec experiments show that preprocessing and model configuration have a measurable impact on performance. Among the tested variants, lemmatization consistently improved the representation, while aggressive stopword removal and alternative pooling strategies such as max pooling or tfidf-weighted mean reduced performance. The best Word2Vec configuration was the lemmatized model with vector_size=500 and window=3, which achieved the highest macro-F1 score among the Word2Vec experiments. Even though the TF-IDF models achieved better macro-F1 and weighted-F1 scores, we selected this model as the final one because the learning curves in the TF-IDF experiments suggested overfitting. For this reason, the Word2Vec model was considered a more stable choice for final prediction. Figure 13.

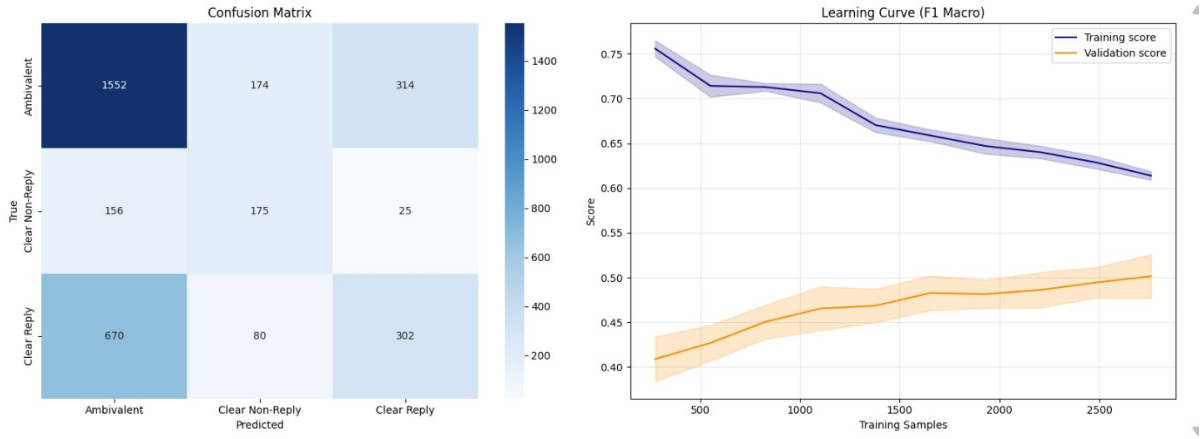


Figure 16: Sample of confusion matrix and learning curve

5.3.3. Conclusion. This study examined the task of clarity classification in political interview responses using two different text representation approaches, namely TF-IDF and Word2Vec, combined with Logistic Regression. Overall, the results show that the problem is particularly challenging. The exploratory analysis showed that the dataset

is dominated by the *Ambivalent* class, while *Clear Non-Reply* is the minority class. Although class imbalance was taken into account during model training and evaluation through stratified cross-validation, macro-F1-based model selection, and the use of class balancing in Logistic Regression, both modeling approaches still had difficulty distinguishing the minority classes. In both TF-IDF and Word2Vec experiments, the models tended to perform better on the majority class, indicating that the imbalance remained an important challenge throughout the study.

The textual analysis also suggested that the classification task is inherently difficult. No particularly strong textual patterns were identified that could clearly separate the classes. The inspection of unigrams, bigrams, and trigrams showed substantial lexical overlap across categories, while the detection of spelling errors and other noisy patterns indicated that the dataset is generally clean and does not require aggressive preprocessing. This helps explain why additional preprocessing steps did not lead to substantial improvements in performance. In other words, the main limitation does not appear to be noise in the text, but rather the lack of strong discriminative linguistic signals that can clearly separate the target classes.

Similarly, the additional analysis of metadata-related factors did not reveal strong external drivers of the labels. The distribution of classes across presidents remained broadly similar, suggesting that the classification patterns are not strongly speaker-dependent. In addition, although response length appeared to differ across classes during exploratory analysis, incorporating answer length as an additional numerical feature did not provide a substantial improvement over the best text-only configuration. This indicates that such information may be mildly correlated with the target labels, but it is not sufficiently informative on its own to significantly improve classification.

Regarding the modeling approaches, TF-IDF achieved the strongest overall predictive performance, with the best results obtained after vectorizer tuning and only marginal differences across several configurations. However, the learning curves consistently suggested overfitting, as the gap between training and validation performance remained noticeable. In contrast, the Word2Vec experiments did not produce better classification scores, but they showed a more stable learning behavior, with a smaller gap between training and validation performance. Nevertheless, this increased stability did not translate into better class discrimination, since the absolute performance remained limited.

Overall, the experimental results indicate that neither feature engineering nor moderate preprocessing changes were sufficient to produce a substantial improvement in this task. Most configurations yielded relatively similar results, and no approach clearly outperformed the others by a large margin. This suggests that the main difficulty of the problem lies where the boundaries between *Clear Reply*, *Clear Non-Reply*, and *Ambivalent* responses are subtle and difficult to capture using relatively simple linear classifiers and standard text representations.

In conclusion, TF-IDF proved to be the more effective representation in terms of predictive performance, while Word2Vec appeared to provide a slightly more stable but not more accurate modeling behavior. Both approaches, however, showed clear limitations in handling this classification task.

6. Bibliography

References

[1] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.

<You should link and cite whatever you used or "got inspired" from the web. Use the / cite command and add the paper/website/tutorial in refs.bib>
<Example of citing a source is like this:> [1] <More about bibtex>